Table of Contents

1. Introduction
    1.1. Scope and Assumptions
2. How To Read This Document
3. Document Health Snapshot
4. Terms and Definitions
    4.1. Diagram Color and Shape Legend
5. Architecture Intent View
    5.1. What This View Answers
    5.2. System Boundary Diagram
    5.3. System Decomposition Diagram
    5.4. Functional Group Manhattan Matrix
    5.5. Functional Unit Dependency Diagram
    5.6. Functional Groups and Units (View Scoped)
6. Communication View
    6.1. What This View Answers
    6.2. Coverage Gaps
    6.3. Recommended Next Evidence Additions
    6.4. Communication Path Graph
    6.5. Functional Groups and Units (View Scoped)
7. Deployment View
    7.1. What This View Answers
    7.2. Coverage Gaps
    7.3. Recommended Next Evidence Additions
    7.4. Deployment Artifact Relationship Graph
    7.5. Platform Operations Graph
    7.6. Functional Groups and Units (View Scoped)
8. Security View
    8.1. What This View Answers
    8.2. Coverage Gaps
    8.3. Recommended Next Evidence Additions
    8.4. Attack Path Diagram
    8.5. Functional Groups and Units (View Scoped)
9. Traceability View
    9.1. What This View Answers
    9.2. Coverage Gaps
    9.3. Recommended Next Evidence Additions
    9.4. Traceability Evidence Graph
    9.5. Requirement-to-Unit Mapping
    9.6. Requirement-to-Unit-to-Evidence Coverage
10. Traceability Appendix
    10.1. Requirement Details
    10.2. Verification Inventory
    10.3. Verification Result Mapping

11. Reference Index
    11.1. Authored References
    11.2. Catalog References
    11.3. Inferred Runtime References
    11.4. Inferred Code References
    11.5. Verification References

---

1. Introduction

This document describes a GitHub App that reviews pull requests using AWS Bedrock and Lambda. The authored functional architecture remains stable while deployment/runtime/code evidence is inferred from infrastructure manifests, source files, and verification artifacts.

1.1. Scope and Assumptions

- Functional Groups and Functional Units are the stable authored design backbone.
- Runtime and code realization are inferred from available IaC, deployment, and source artifacts.
- Inferred coverage is evidence-driven and may be partial when source metadata is missing.
- Requirement alignment and reference indexes are provided for traceability, not as primary narrative flow.

2. How To Read This Document

This document is intentionally split into two layers:

1. Concept layer: functional and view-specific architecture narratives for human understanding.
2. Evidence layer: generated requirement traceability and inferred reference indexes.

Read in this order:

1. Architecture Intent View: stable authored architecture intent and ownership boundaries.
2. Communication / Deployment / Security Views: viewpoint-specific operational behavior.
3. Traceability View: requirement mapping, evidence coverage, and confidence.
4. Reference Appendix: supporting reference IDs and inferred-source links.

3. Document Health Snapshot

View	Authored Status	Authored Status Explanation	Units in Scope	Direct Evidence Coverage	Gap Count	Heuristic Confidence	Heuristic Basis
Architecture Intent View (`VIEW-ARCHITECTURE-INTENT`)	stable	Authored capability boundaries and ownership responsibilities are stable for this release.	7	7	0	high	Defaults authored units to covered; use authoredStatus as the primary publication signal.
Communication View (`VIEW-COMMUNICATION`)	in-review	Communication sequencing is defined, while protocol-level metadata is still being expanded.	7	3	4	low	Counts a unit as covered when direct inferred runtime evidence exists.
Deployment View (`VIEW-DEPLOYMENT`)	in-review	Deployment and runtime mapping are present; observability and ownership evidence are still evolving.	7	3	4	low	Counts a unit as covered when direct inferred runtime evidence exists.
Security View (`VIEW-SECURITY`)	draft	Core threats are modeled; control-depth and detection mappings are still being expanded.	7	5	2	medium	Counts a unit as covered when explicit authored attack-vector mapping exists.
Traceability View (`VIEW-TRACEABILITY`)	in-review	Requirement mappings exist; evidence completeness and consistency are still being hardened.	7	7	4	high	Counts a unit as covered when direct inferred runtime or code evidence exists.

4. Terms and Definitions

Term	Definition
actor (EM-ACTOR)	A person or operational role that interacts with functional units.
ai review is enabled (FEAT-AI-REVIEW)	Feature flag enabling model-assisted review. Aliases:ai review enabled
attack vector (EM-ATTACK-VECTOR)	A technical misuse or attack path that targets functional, referenced, or runtime elements.
audit logging is enabled (FEAT-AUDIT-LOGGING)	Feature flag enabling audit metadata persistence. Aliases:audit enabled
authored mapping (EM-AUTHORED-MAPPING)	An explicit relationship declared in architecture inputs between authored or referenced elements.
aws sdk for go (REF-AWS-SDK-GO)	SDK dependency used for AWS runtime integrations. Aliases:aws sdk
aws secrets manager (REF-AWS-SECRETS-MANAGER)	Managed secret store for application credentials and config values. Aliases:secrets manager
bedrock runtime (REF-AWS-BEDROCK-RUNTIME)	External Bedrock runtime endpoint used for model inference. Aliases:bedrock api
bedrock runtime is unavailable (EVT-BEDROCK-UNAVAILABLE)	Event raised when Bedrock inference path is degraded. Aliases:bedrock unavailable
check run summary (DATA-CHECKRUN-SUMMARY)	Aggregated review result posted as a GitHub check run.
code element (EM-CODE-ELEMENT)	An inferred code structure or ownership element discovered from source trees and build metadata.
developer (ACT-DEVELOPER)	Engineer opening and updating pull requests. Aliases:pr author
diff context (DATA-DIFF-CONTEXT)	Pull request file changes and metadata prepared for analysis.
diff context is prepared (EVT-CONTEXT-PREPARED)	Event indicating context assembly completed and is ready for analysis. Aliases:context prepared
functional group (EM-FUNCTIONAL-GROUP)	A major authored capability area that groups related functional units.
functional unit (EM-FUNCTIONAL-UNIT)	An authored working unit inside a functional group that owns specific behavior.
github app api (REF-GITHUB-APP-API)	External GitHub API used for pull request metadata and review publication. Aliases:github api

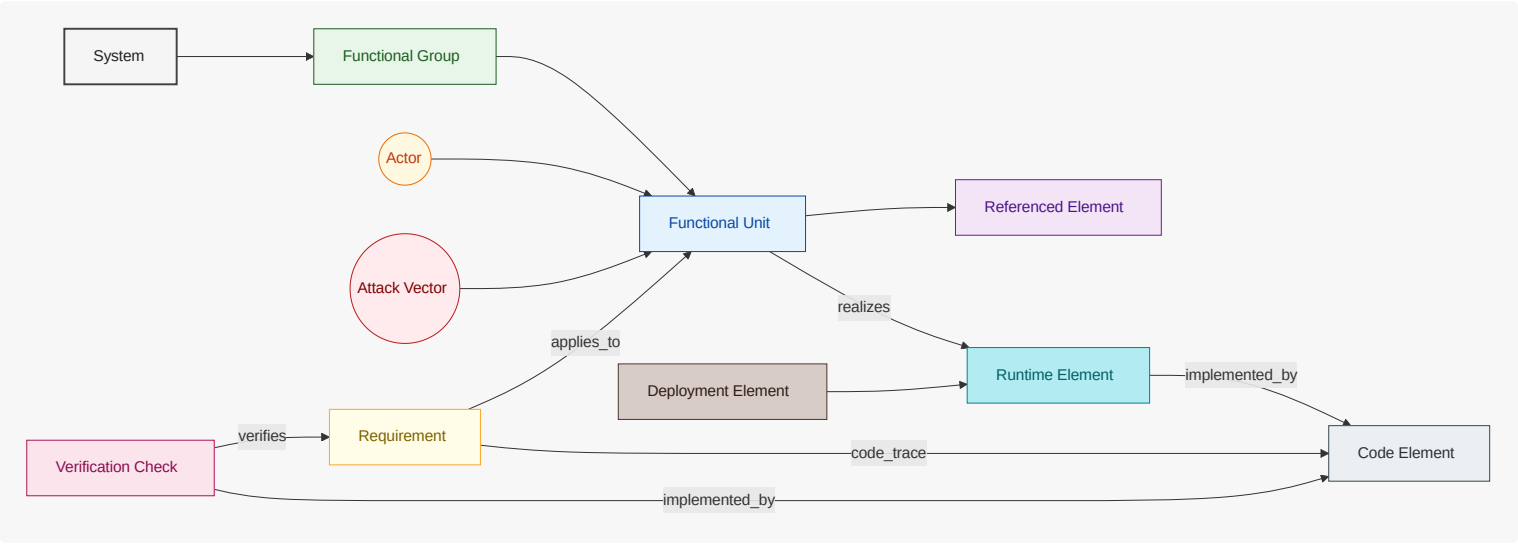
Term	Definition
github rest sdk (REF-GITHUB-REST-SDK)	SDK dependency used for GitHub App REST interactions. Aliases:github sdk
github webhook ingress (FU-GITHUB-WEBHOOK-INGRESS)	Validates webhook signatures and routes trusted pull request events. Aliases:webhook ingress
inferred mapping (EM-INFERRED-MAPPING)	A discovered relationship that links inferred runtime/code elements upward to authored design.
lambda runtime operations (FU-LAMBDA-RUNTIME-OPERATIONS)	Manages runtime deployment and operational controls. Aliases:lambda ops
normal mode is active (MODE-NORMAL)	Default operating mode using AI plus deterministic checks. Aliases:normal mode
platform operations (FG-PLATFORM-OPERATIONS)	Functional group for deployment lifecycle and secret/config controls. Aliases:runtime operations
platform operator (ACT-PLATFORM-OPERATOR)	Owner of deployment/runtime operations and reliability controls. Aliases:platform ops
policy checks (FU-POLICY-CHECKS)	Evaluates deterministic repository policy checks. Aliases:deterministic checks
policy checks are enabled (FEAT-POLICY-CHECKS)	Feature flag enabling deterministic policy checks. Aliases:policy checks enabled
policy ruleset (REF-POLICY-RULESET)	Deterministic review policy bundle used during analysis. Aliases:policy bundle
pr context assembly (FU-PR-CONTEXT-ASSEMBLY)	Fetches changed files and prepares bounded review context payloads. Aliases:context assembly
pr integration (FG-PR-INTEGRATION)	Functional group for webhook ingress and pull request context intake. Aliases:github integration
prompt injection in diff (AV-PROMPT-INJECTION-IN-DIFF)	Malicious prompt-like content embedded in pull request code changes. Aliases:prompt injection
pull request opened event is received (EVT-PR-OPENED)	Event raised when a new pull request is opened. Aliases:pr opened
pull request review is active (STATE-PR-REVIEW-ACTIVE)	State where review processing is currently active for a pull request. Aliases:review active
pull request spam abuse (AV-PR-SPAM-ABUSE)	High-volume event abuse causing review capacity exhaustion. Aliases:pr spam

Term	Definition
pull request synchronize event is received (EVT-PR-SYNCHRONIZE)	Event raised when new commits are pushed to an open pull request. Aliases:pr synchronize
redacted audit metadata (DATA-REDACTED-AUDIT-METADATA)	Sanitized prompt and finding metadata retained for audit.
referenced element (EM-REFERENCED-ELEMENT)	An architecture-relevant external, platform, or third-party dependency represented by role.
repository maintainer (ACT-REPOSITORY-MAINTAINER)	Owner of merge decisions and repository quality gates. Aliases:maintainer
result metadata (DATA-RESULT-METADATA)	Metadata describing review output characteristics for audit and reporting.
review findings are generated (EVT-FINDINGS-GENERATED)	Event indicating analysis output is ready for publication. Aliases:findings generated
review findings are published (EVT-FINDINGS-PUBLISHED)	Event indicating review output has been published to GitHub checks/comments. Aliases:findings published
review intelligence (FG-REVIEW-INTELLIGENCE)	Functional group for AI and deterministic review analysis and publication. Aliases:pr review system
review orchestration (FU-REVIEW-ORCHESTRATION)	Coordinates Bedrock analysis with deterministic checks and fallback logic. Aliases:orchestration
review publication (FU-REVIEW-PUBLICATION)	Publishes review findings to GitHub checks and comments. Aliases:publication
review request rate limit is exceeded (EVT-RATE-LIMIT-EXCEEDED)	Event raised when PR review rate guardrails are hit. Aliases:rate limit exceeded
runtime element (EM-RUNTIME-ELEMENT)	An inferred runtime realization element discovered from infrastructure and deployment sources.
secret leakage in review output (AV-SECRET-LEAKAGE-IN-REVIEW)	Unintended credential disclosure in generated review findings. Aliases:secret leakage
secrets and configuration (FU-SECRETS-CONFIGURATION)	Manages credentials and secure runtime configuration. Aliases:secret config
security engineer (ACT-SECURITY-ENGINEER)	Owner of security policy checks and finding triage. Aliases:security reviewer
spoofed github webhook (AV-SPOOFED-GITHUB-WEBHOOK)	Forged webhook request attempting unauthorized review execution. Aliases:webhook spoofing

Term	Definition
upward linking (EM-UPWARD-LINKING)	Rule where runtime and code elements point to stable functional groups/units; authored architecture does not depend on inferred IDs.
webhook signature (DATA-WEBHOOK-SIGNATURE)	Signature used to verify webhook authenticity.
webhook signature is valid (COND-WEBHOOK-SIGNATURE-VALID)	Condition indicating webhook HMAC verification succeeded. Aliases:signature valid

4.1. Diagram Color and Shape Legend

This legend applies to all Mermaid diagrams in this document.



5. Architecture Intent View

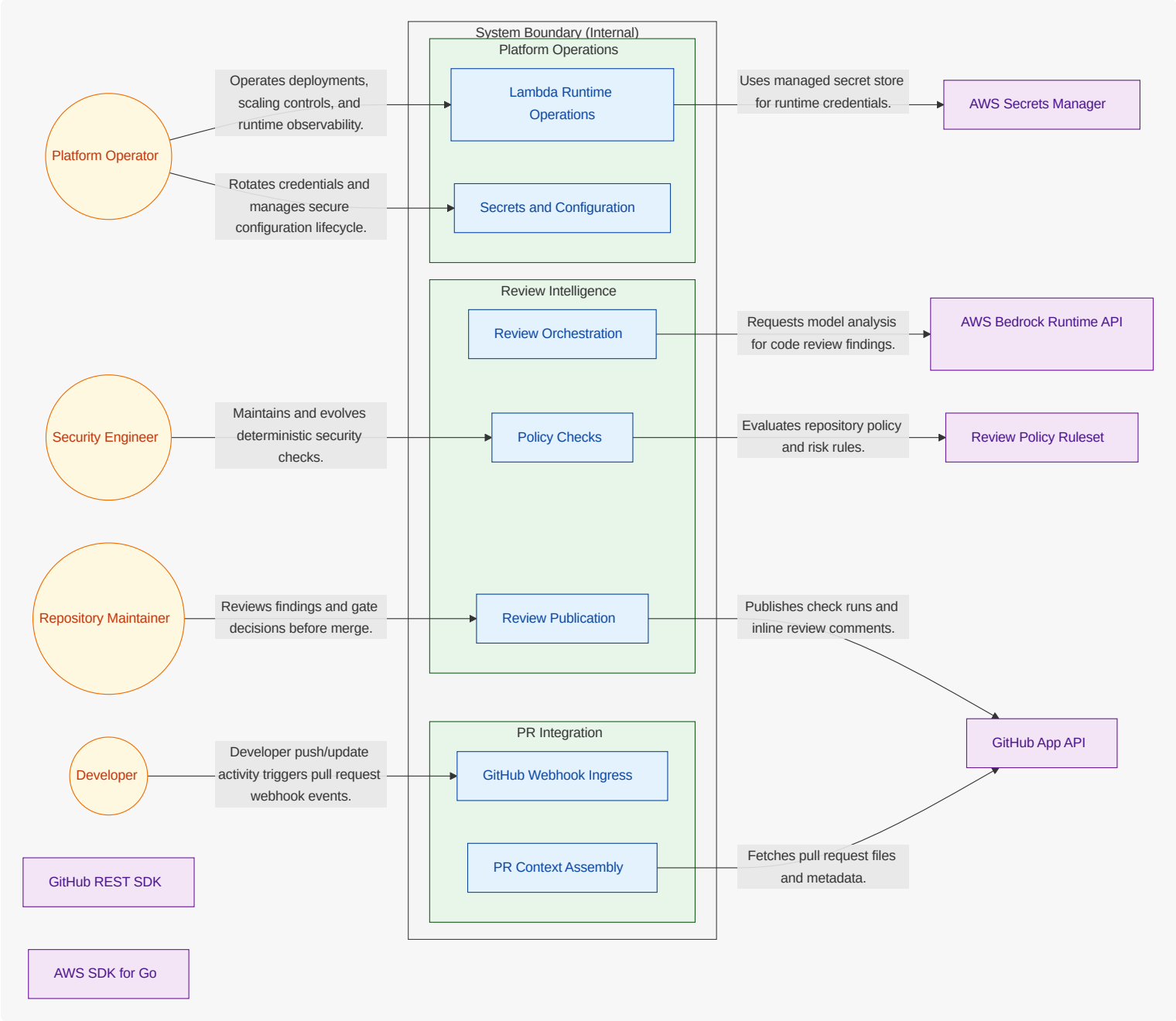
Show the stable authored architecture intent: capability areas, unit ownership boundaries, and intentional responsibility split.

5.1. What This View Answers

- What are the main capability areas?
- What are the stable units of responsibility and what does each unit own?
- How are responsibilities intentionally split?

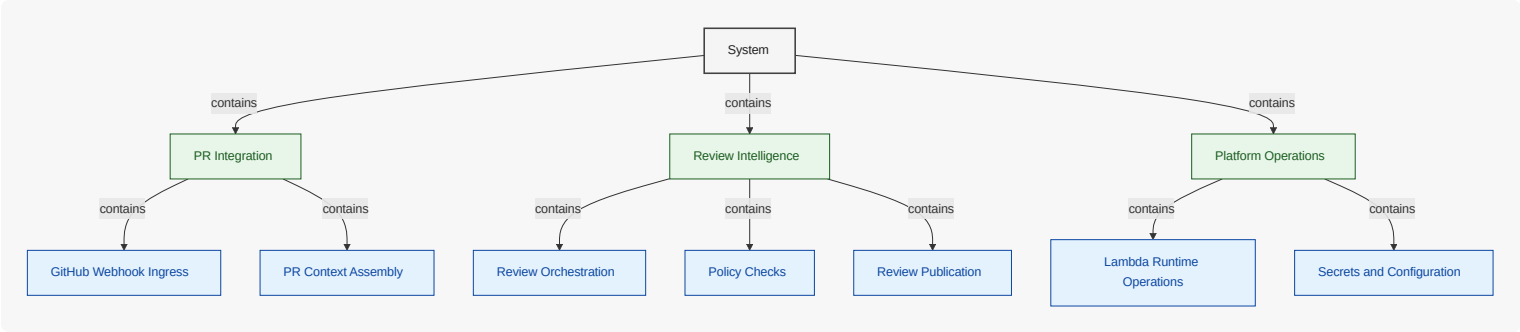
5.2. System Boundary Diagram

What this diagram shows: explicit internal vs external scope. Functional units are inside the `System Boundary (Internal)` container, while actors and referenced dependencies are shown outside the boundary.



5.3. System Decomposition Diagram

What this diagram shows: stable capability decomposition from functional groups into functional units.



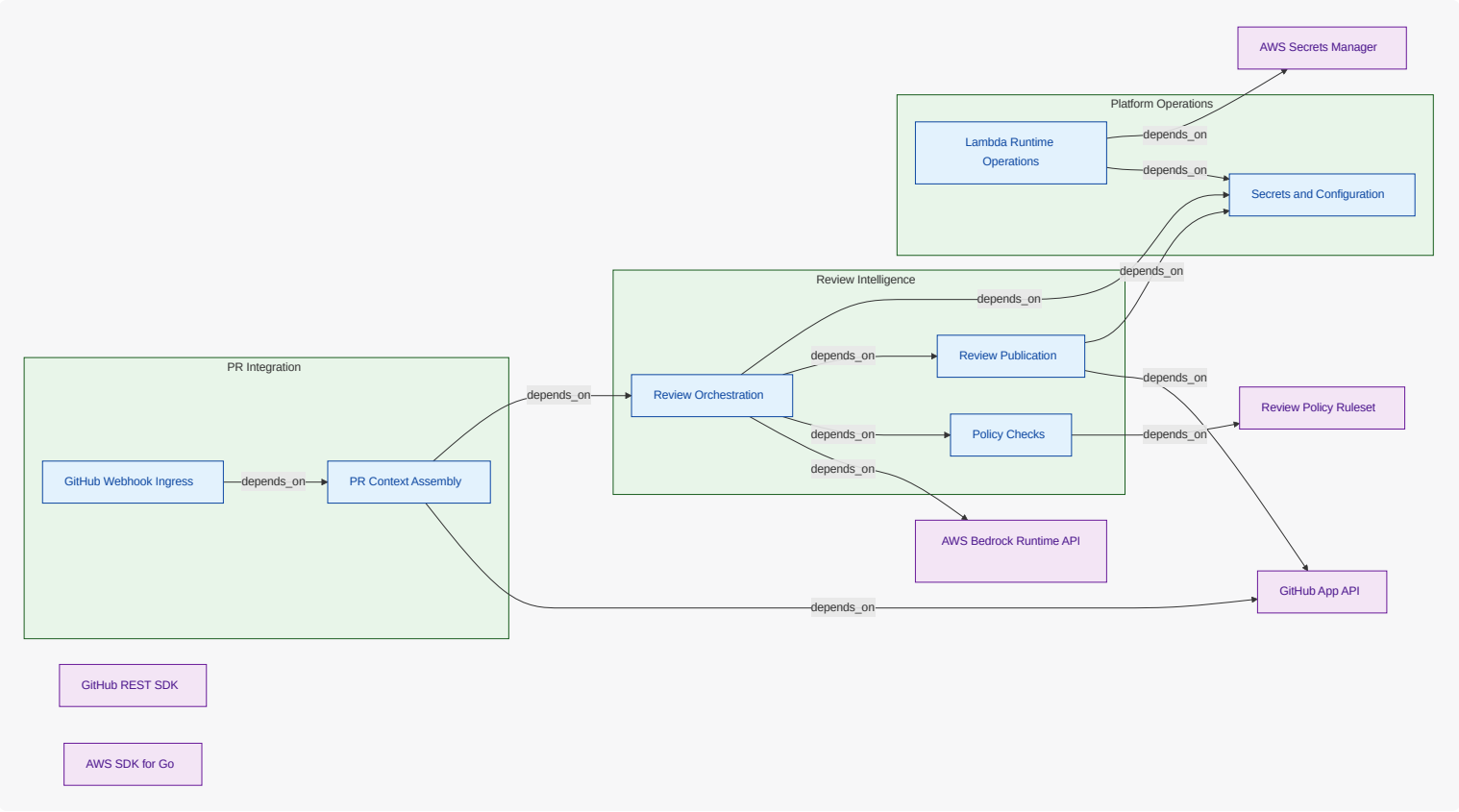
5.4. Functional Group Manhattan Matrix

What this diagram shows: a table-like matrix where functional groups are the bottom row (columns) and functional units are arranged in rows above their owning group.

	Policy Checks	
GitHub Webhook Ingress	Review Orchestration	Lambda Runtime Operations
PR Context Assembly	Review Publication	Secrets and Configuration
PR Integration	Review Intelligence	Platform Operations

5.5. Functional Unit Dependency Diagram

What this diagram shows: dependency paths between functional units (grouped in Functional Group containers) and key referenced elements. Arrow meaning: `A -→|depends_on| B` means unit `A` requires behavior or data from `B` to fulfill its responsibility.



5.6. Functional Groups and Units (View Scoped)

5.6.1. PR Integration (FG-PR-INTEGRATION)

Ingress and context assembly for GitHub pull request events.

PR Integration owns webhook trust boundaries and pull request context intake. It converts incoming GitHub events into normalized review work with clear boundaries.

GitHub Webhook Ingress (FU-GITHUB-WEBHOOK-INGRESS)

Verifies webhook signatures, validates event type, and starts review workflow routing.

Owned Responsibility

functional responsibility for github webhook ingress; includes decision and orchestration flow to PR Context Assembly

Key Collaborators

PR Context Assembly

Linked Requirements

REQ-PRR-001, REQ-PRR-008

PR Context Assembly (FU-PR-CONTEXT-ASSEMBLY)

Fetches changed files and diff metadata and prepares bounded review context payloads.

Owned Responsibility

functional responsibility for pr context assembly; includes decision and orchestration flow to GitHub App API and decision and orchestration flow to Review Orchestration

Key Collaborators

GitHub App API, Review Orchestration

Linked Requirements

REQ-PRR-002

5.6.2. Review Intelligence (FG-REVIEW-INTELLIGENCE)

AI and deterministic review analysis with result publication.

Review Intelligence owns generated findings, deterministic checks, and final publication behavior. It combines Bedrock reasoning with predictable policy checks and emits auditable review outputs.

Policy Checks (FU-POLICY-CHECKS)

Runs deterministic checks for secrets, risky IaC changes, and repository policy constraints.

Owned Responsibility

functional responsibility for policy checks; includes decision and orchestration flow to Review Policy Ruleset

Key Collaborators

Review Policy Ruleset

Linked Requirements

REQ-PRR-004, REQ-PRR-006

Review Orchestration (FU-REVIEW-ORCHESTRATION)

Calls Bedrock models, merges AI and deterministic findings, and handles graceful degradation paths.

Owned Responsibility

functional responsibility for review orchestration; includes decision and orchestration flow to AWS Bedrock Runtime API and decision and orchestration flow to Policy Checks

Key Collaborators

AWS Bedrock Runtime API, Policy Checks, Review Publication, Secrets and Configuration

Linked Requirements

REQ-PRR-003, REQ-PRR-006

Review Publication (FU-REVIEW-PUBLICATION)

Publishes check-run summaries and inline comments back to GitHub pull requests.

Owned Responsibility

functional responsibility for review publication; includes decision and orchestration flow to GitHub App API and decision and orchestration flow to Secrets and Configuration

Key Collaborators

GitHub App API, Secrets and Configuration

Linked Requirements

REQ-PRR-005, REQ-PRR-006, REQ-PRR-007, REQ-PRR-008

5.6.3. Platform Operations (FG-PLATFORM-OPERATIONS)

Lambda deployment lifecycle, runtime operations, and secrets/configuration.

Platform Operations owns safe deployment, runtime controls, and secret/config lifecycle for the app. It enables dependable review operation without owning business-level review policy semantics.

Lambda Runtime Operations (FU-LAMBDA-RUNTIME-OPERATIONS)

Manages Lambda packaging/deployment lifecycle and operational health guardrails.

Owned Responsibility

functional responsibility for lambda runtime operations; includes decision and orchestration flow to AWS Secrets Manager and decision and orchestration flow to Secrets and Configuration

Key Collaborators

AWS Secrets Manager, Secrets and Configuration

Linked Requirements

no explicit authored requirement trigger

Secrets and Configuration (FU-SECRETS-CONFIGURATION)

Manages GitHub App credentials, Bedrock config, and redaction-safe runtime configuration.

Owned Responsibility

functional responsibility for secrets and configuration

Key Collaborators

none

Linked Requirements

REQ-PRR-007

6. Communication View

Show who communicates with whom across key request paths, including interface edges and dependency handoffs.

6.1. What This View Answers

- Who talks to whom and over which interfaces?
- Which edges are synchronous, asynchronous, event-driven, or callback-oriented?
- Which communication paths are critical to the main request flow?

6.2. Coverage Gaps

Coverage summary: 3/7 units have direct evidence coverage in this view.

- Lambda Runtime Operations has no direct inferred runtime/deployment evidence yet.
- PR Context Assembly has no direct inferred runtime/deployment evidence yet.
- Policy Checks has no direct inferred runtime/deployment evidence yet.
- Secrets and Configuration has no direct inferred runtime/deployment evidence yet.

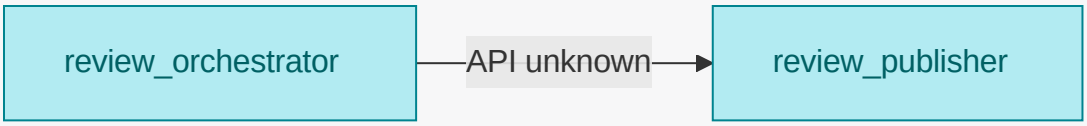
6.3. Recommended Next Evidence Additions

- Add explicit communication metadata (protocol, sync/async/event/callback) to authored mappings where currently implicit.
- Expose service protocol/port metadata in Helm or manifest values to enrich interface-edge inference.
- Capture missing communicating runtime units by adding discoverable workload artifacts under scanned inference roots.

6.4. Communication Path Graph

What this diagram shows: inferred unit-to-unit request/call flow, including available interface/port labels.

> **NOTE** This graph is inferred from available deployment/runtime artifacts and may be partial. Missing edges or interface details indicate missing derived communication evidence, not necessarily missing behavior.



6.5. Functional Groups and Units (View Scoped)

6.5.1. PR Integration (FG-PR-INTEGRATION)

PR Integration receives webhook events and forwards normalized review context to orchestration with explicit handoff semantics.

GitHub Webhook Ingress (FU-GITHUB-WEBHOOK-INGRESS)

Receives webhook payloads, validates them, and routes authorized events to context assembly.

Inbound Interfaces

Interface	Description
Developer	Developer push/update activity triggers pull request webhook events.

Outbound Interfaces

Interface	Description
PR Context Assembly	Forwards authenticated PR events for context assembly.

Communication Evidence

runtime: webhook_ingress | code modules: webhook_ingress

PR Context Assembly (FU-PR-CONTEXT-ASSEMBLY)

Calls GitHub APIs and forwards normalized context to orchestration.

Inbound Interfaces

Interface	Description
GitHub Webhook Ingress	Forwards authenticated PR events for context assembly.

Outbound Interfaces

Interface	Description
GitHub App API	Fetches pull request files and metadata.
Review Orchestration	Sends normalized review context.

Communication Evidence

code modules: pr_context_assembly

6.5.2. Review Intelligence (FG-REVIEW-INTELLIGENCE)

Review Intelligence coordinates Bedrock calls, policy evaluation, and publication sequencing for deterministic outcomes.

Policy Checks (FU-POLICY-CHECKS)

Exchanges policy findings with orchestration for final result composition.

Inbound Interfaces

Interface	Description
Review Orchestration	Merges deterministic policy findings with model output.
Security Engineer	Maintains and evolves deterministic security checks.

Outbound Interfaces

Interface	Description
Review Policy Ruleset	Evaluates repository policy and risk rules.

Communication Evidence

code modules: policy_checks

Review Orchestration (FU-REVIEW-ORCHESTRATION)

Integrates context input, Bedrock calls, and publication output in a deterministic sequence.

Inbound Interfaces

Interface	Description
PR Context Assembly	Sends normalized review context.

Outbound Interfaces

Interface	Description
AWS Bedrock Runtime API	Requests model analysis for code review findings.
Policy Checks	Merges deterministic policy findings with model output.
Review Publication	Submits final findings for publication.
Secrets and Configuration	Retrieves model configuration and safety settings.

Communication Evidence

runtime: review_orchestrator | code modules: review_orchestrator

Review Publication (FU-REVIEW-PUBLICATION)

Posts summaries and inline annotations through GitHub App APIs.

Inbound Interfaces

Interface	Description
Repository Maintainer	Reviews findings and gate decisions before merge.
Review Orchestration	Submits final findings for publication.

Outbound Interfaces

Interface	Description
GitHub App API	Publishes check runs and inline review comments.
Secrets and Configuration	Retrieves app credentials and redaction rules.

Communication Evidence

runtime: review_publisher | code modules: review_publication

6.5.3. Platform Operations (FG-PLATFORM-OPERATIONS)

Platform Operations coordinates runtime controls with app units via deployment and secret/config integration interfaces.

Lambda Runtime Operations (FU-LAMBDA-RUNTIME-OPERATIONS)

Operates deployment and runtime controls that indirectly shape request-path reliability.

Inbound Interfaces

Interface	Description
Platform Operator	Operates deployments, scaling controls, and runtime observability.

Outbound Interfaces

Interface	Description
AWS Secrets Manager	Uses managed secret store for runtime credentials.
Secrets and Configuration	Requires secure runtime configuration and credential access.

Communication Evidence

code modules: lambda_runtime_ops

Secrets and Configuration (FU-SECRETS-CONFIGURATION)

Provides secure config retrieval contracts to orchestration and publication units.

Inbound Interfaces

Interface	Description
Lambda Runtime Operations	Requires secure runtime configuration and credential access.
Platform Operator	Rotates credentials and manages secure configuration lifecycle.
Review Orchestration	Retrieves model configuration and safety settings.
Review Publication	Retrieves app credentials and redaction rules.

Outbound Interfaces

none

Communication Evidence

code modules: secrets_configuration

7. Deployment View

Show inferred deployment artifacts and ownership mapping to authored units. This view emphasizes deployment relationships and platform operations.

7.1. What This View Answers

- How does authored intent become deployed runtime?
- What control-plane objects manage deployable units?
- Where are platform ownership boundaries and missing ownership signals?

7.2. Coverage Gaps

Coverage summary: 3/7 units have direct evidence coverage in this view.

- Lambda Runtime Operations has no direct inferred runtime/deployment evidence yet.
- PR Context Assembly has no direct inferred runtime/deployment evidence yet.
- Policy Checks has no direct inferred runtime/deployment evidence yet.
- Secrets and Configuration has no direct inferred runtime/deployment evidence yet.

7.3. Recommended Next Evidence Additions

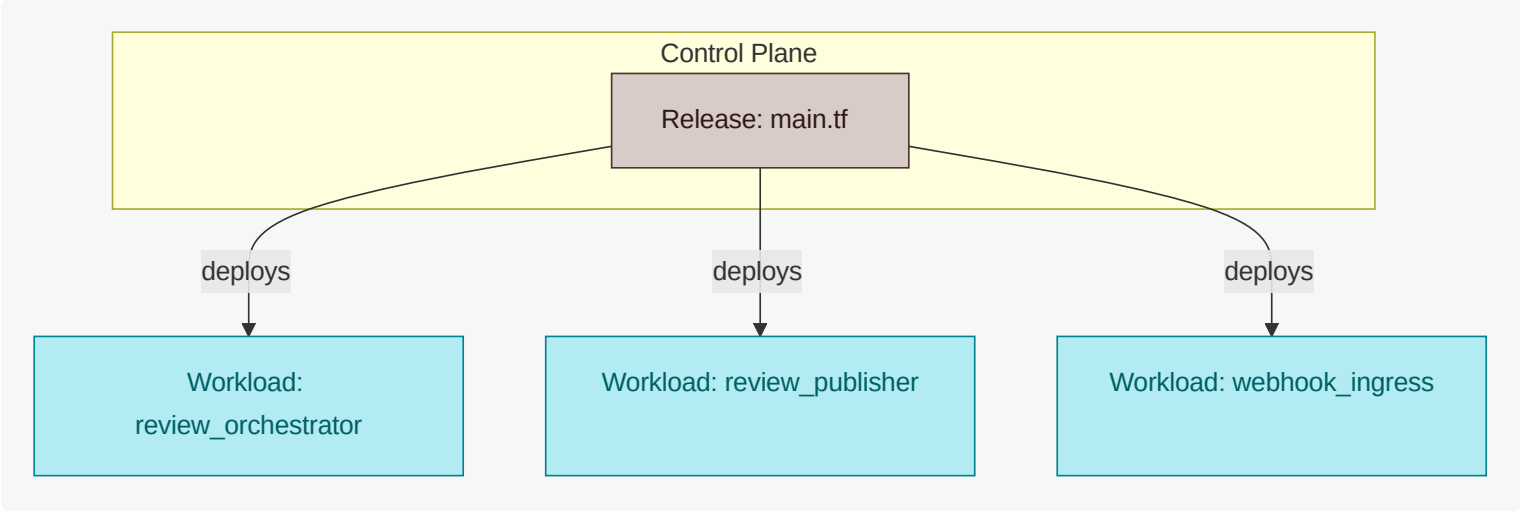
- Add owner metadata to Flux/Kustomization/Helm objects for clearer platform ownership mapping.
- Ensure deployment control artifacts (source, kustomization, release) are all included in scanned directories.
- Add deterministic naming conventions for release and namespace objects to improve cross-artifact linking.

7.4. Deployment Artifact Relationship Graph

What this diagram shows: how source/control-plane artifacts connect to releases, namespaces, workloads, and target cluster context.

NOTE

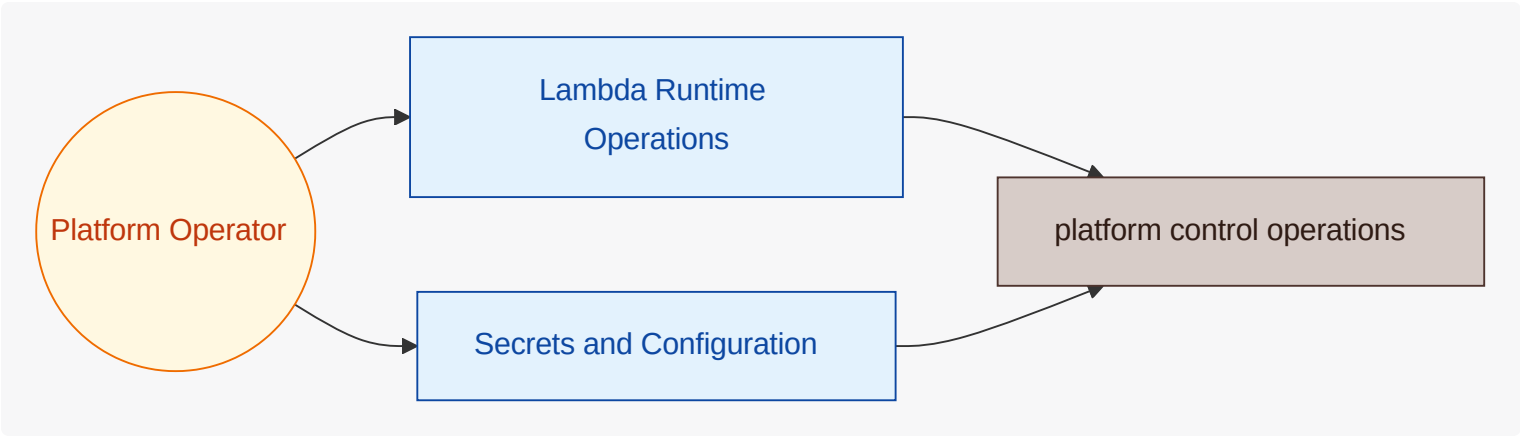
This graph is inferred from available deployment artifacts (for example Terraform, Flux, Helm) and may be partial when source metadata is incomplete. Namespace and cluster boundaries are rendered as containers when target/placement relationships are inferred. Node labels include role hints (for example `Source`, `Kustomization`, `Release`, `Workload`, `ns/…`) to distinguish similarly named elements.



7.5. Platform Operations Graph

What this diagram shows: which platform actors operate which platform units and the control-plane targets they influence.

> **NOTE** Platform operations links are inferred from authored actor mappings plus discovered deployment control-plane artifacts.



7.6. Functional Groups and Units (View Scoped)

7.6.1. PR Integration (FG-PR-INTEGRATION)

PR Integration functions deploy independently to isolate webhook ingress from context-fetch implementation changes.

GitHub Webhook Ingress (FU-GITHUB-WEBHOOK-INGRESS)

GitHub Webhook Ingress Deployment

Deploys as an independently managed Lambda runtime boundary.

Deployment Aspect	View-Scoped Detail
Deployment trigger context	REQ-PRR-001, REQ-PRR-008
Deployment dependencies	PR Context Assembly

Deployment Aspect	View-Scoped Detail
Inferred deployment evidence	runtime: webhook_ingress

PR Context Assembly (FU-PR-CONTEXT-ASSEMBLY)

PR Context Assembly Deployment

Deployed separately to allow context shaping changes without ingress disruption.

Deployment Aspect	View-Scoped Detail
Deployment trigger context	REQ-PRR-002
Deployment dependencies	GitHub App API, Review Orchestration
Inferred deployment evidence	code modules: pr_context_assembly

7.6.2. Review Intelligence (FG-REVIEW-INTELLIGENCE)

Review Intelligence units roll independently to preserve availability and reduce blast radius during model or rule updates.

Policy Checks (FU-POLICY-CHECKS)

Policy Checks Deployment

Deployed independently to evolve policy logic without model coupling.

Deployment Aspect	View-Scoped Detail
Deployment trigger context	REQ-PRR-004, REQ-PRR-006
Deployment dependencies	Review Policy Ruleset
Inferred deployment evidence	code modules: policy_checks

Review Orchestration (FU-REVIEW-ORCHESTRATION)

Review Orchestration Deployment

Isolated deployment enables cautious model/config rollout and fallback control.

Deployment Aspect	View-Scoped Detail
Deployment trigger context	REQ-PRR-003, REQ-PRR-006
Deployment dependencies	AWS Bedrock Runtime API, Policy Checks, Review Publication, Secrets and Configuration
Inferred deployment evidence	runtime: review_orchestrator

Review Publication (FU-REVIEW-PUBLICATION)

Review Publication Deployment

Release isolation protects publication reliability during other unit rollouts.

Deployment Aspect	View-Scoped Detail
Deployment trigger context	REQ-PRR-005, REQ-PRR-006, REQ-PRR-007, REQ-PRR-008
Deployment dependencies	GitHub App API, Secrets and Configuration
Inferred deployment evidence	runtime: review_publisher

7.6.3. Platform Operations (FG-PLATFORM-OPERATIONS)

Platform Operations governs release orchestration and runtime placement to keep review capacity stable.

Lambda Runtime Operations (FU-LAMBDA-RUNTIME-OPERATIONS)

Lambda Runtime Operations Deployment

Governs source-to-release pipeline behavior and runtime placement boundaries.

Deployment Aspect	View-Scoped Detail
Deployment trigger context	no explicit authored requirement trigger
Deployment dependencies	AWS Secrets Manager, Secrets and Configuration
Inferred deployment evidence	code modules: lambda_runtime_ops

Secrets and Configuration (FU-SECRETS-CONFIGURATION)

Secrets and Configuration Deployment

Managed as a shared operational boundary with controlled rotation and rollout flow.

Deployment Aspect	View-Scoped Detail
Deployment trigger context	REQ-PRR-007
Deployment dependencies	none
Inferred deployment evidence	code modules: secrets_configuration

8. Security View

Show inferred exposure and dependency risk points aligned to unit boundaries, focused on attack paths and security evidence.

8.1. What This View Answers

- What are the main trust boundaries and attack paths?
- Which units are exposed and what security evidence exists?
- Where is the threat model incomplete?

8.2. Coverage Gaps

Coverage summary: 5/7 units have direct evidence coverage in this view.

- Lambda Runtime Operations has no explicit authored attack-vector mapping yet; document technical and fraud/abuse threats even when mitigated.
- Policy Checks has no explicit authored attack-vector mapping yet; document technical and fraud/abuse threats even when mitigated.

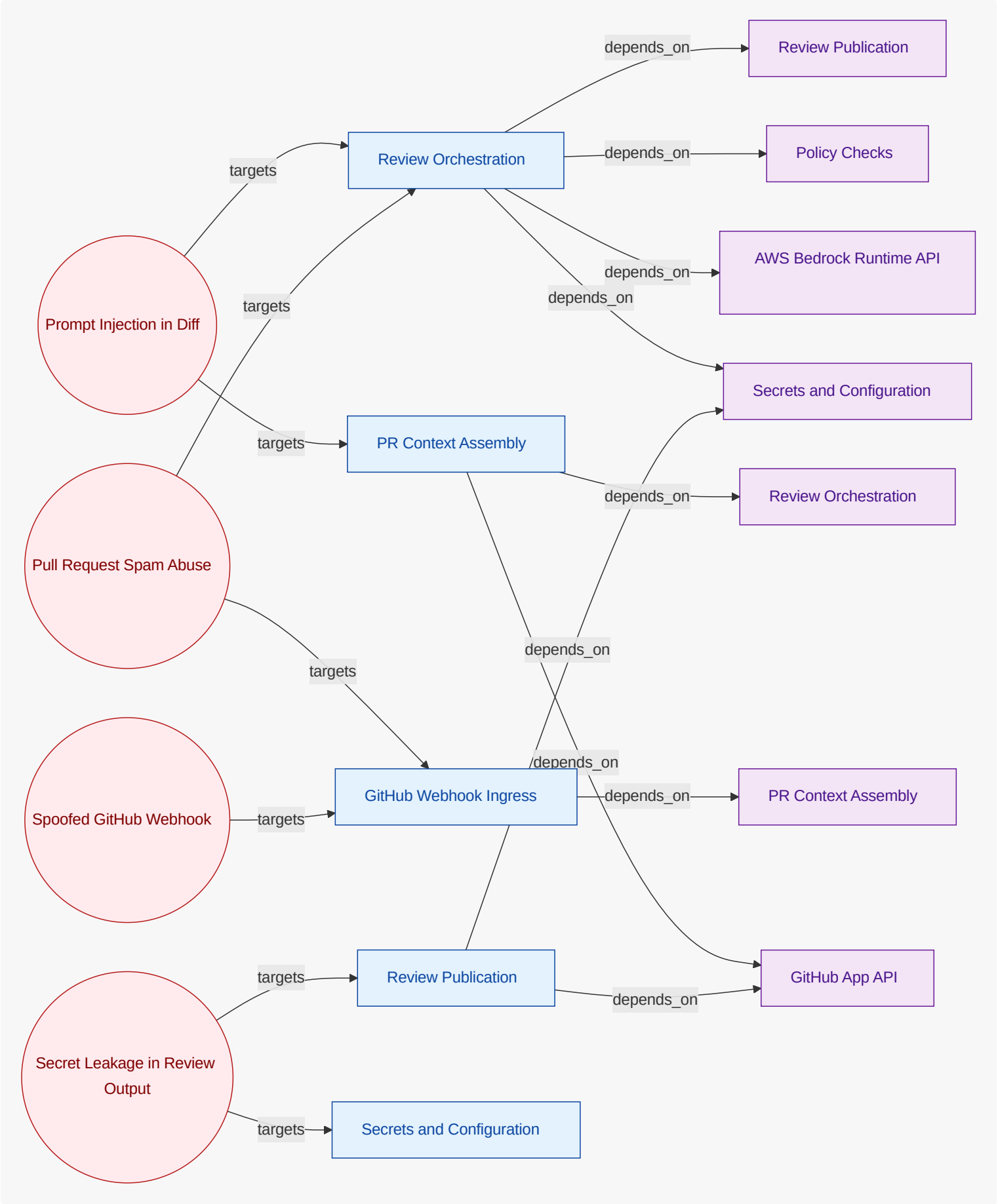
8.3. Recommended Next Evidence Additions

- Add explicit authored attack-vector mappings for units currently marked without threat linkage.
- Add security signal ownership metadata (logs/alerts/events) for unresolved observability evidence.
- Map additional security-relevant dependencies and exposures into authored mappings for traceable coverage.

8.4. Attack Path Diagram

What this diagram shows: authored attack vectors, their targeted units, and security-relevant dependency paths.

> **NOTE** Security relationships shown here are explicitly modeled or inferred from available evidence and may be incomplete. Absence of a path does not imply absence of risk. Threats that are currently mitigated (including fraud/abuse threats) should still be modeled as attack vectors for auditability and control-trace visibility.


Prompt Injection in Diff
Pull Request Spam Abuse
Spoofed GitHub Webhook
Secret Leakage in Review Output
Review Orchestration
PR Context Assembly
GitHub Webhook Ingress
Review Publication
Secrets and Configuration
Review Publication
Policy Checks
AWS Bedrock Runtime API
Secrets and Configuration
Review Orchestration
PR Context Assembly
GitHub App API
targets
targets
targets
targets
targets
targets
targets
depends_on
depends_on
depends_on
depends_on
depends_on
depends_on
depends_on
depends_on
depends_on

8.5. Functional Groups and Units (View Scoped)

8.5.1. PR Integration (FG-PR-INTEGRATION)

PR Integration protects signature verification and hostile-diff ingress paths to reduce spoofing and prompt injection risk.

GitHub Webhook Ingress (FU-GITHUB-WEBHOOK-INGRESS)

Enforces signature verification and rejects spoofed or malformed events.

Attack Surface

PR Context Assembly

Main Threats

Pull Request Spam Abuse, Spoofed GitHub Webhook

Expected Controls

Define control expectations in authored security narratives and mapped dependencies.

Security Evidence Present

runtime: webhook_ingress | code modules: webhook_ingress

Security Evidence Missing

Close any unresolved ownership or unmapped threat-linkage gaps in Traceability View.

PR Context Assembly (FU-PR-CONTEXT-ASSEMBLY)

Applies contextual sanitization to reduce prompt injection risk from diffs.

Attack Surface

GitHub App API, Review Orchestration

Main Threats

Prompt Injection in Diff

Expected Controls

Define control expectations in authored security narratives and mapped dependencies.

Security Evidence Present

code modules: pr_context_assembly

Security Evidence Missing

Close any unresolved ownership or unmapped threat-linkage gaps in Traceability View.

8.5.2. Review Intelligence (FG-REVIEW-INTELLIGENCE)

Review Intelligence emphasizes prompt hardening, output redaction, and secure publication boundaries.

Policy Checks (FU-POLICY-CHECKS)

Provides deterministic controls for secrets and risky infrastructure changes.

Attack Surface

Review Policy Ruleset

Main Threats

no explicit attack vector targeting this unit

Expected Controls

Define control expectations in authored security narratives and mapped dependencies.

Security Evidence Present

code modules: policy_checks

Security Evidence Missing

Close any unresolved ownership or unmapped threat-linkage gaps in Traceability View.

Review Orchestration (FU-REVIEW-ORCHESTRATION)

Enforces safe prompt construction and bounded output handling under adversarial input.

Attack Surface

AWS Bedrock Runtime API, Policy Checks, Review Publication, Secrets and Configuration

Main Threats

Prompt Injection in Diff, Pull Request Spam Abuse

Expected Controls

Define control expectations in authored security narratives and mapped dependencies.

Security Evidence Present

runtime: review_orchestrator | code modules: review_orchestrator

Security Evidence Missing

Close any unresolved ownership or unmapped threat-linkage gaps in Traceability View.

Review Publication (FU-REVIEW-PUBLICATION)

Applies redaction and policy-aware output filtering before publishing findings.

Attack Surface

GitHub App API, Secrets and Configuration

Main Threats

Secret Leakage in Review Output

Expected Controls

Define control expectations in authored security narratives and mapped dependencies.

Security Evidence Present

runtime: review_publisher | code modules: review_publication

Security Evidence Missing

Close any unresolved ownership or unmapped threat-linkage gaps in Traceability View.

8.5.3. Platform Operations (FG-PLATFORM-OPERATIONS)

Platform Operations secures credential paths and operational controls across Lambda deployment lifecycle.

Lambda Runtime Operations (FU-LAMBDA-RUNTIME-OPERATIONS)

Hardens operational control loops and protects deployment trust boundaries.

Attack Surface

AWS Secrets Manager, Secrets and Configuration

Main Threats

no explicit attack vector targeting this unit

Expected Controls

Define control expectations in authored security narratives and mapped dependencies.

Security Evidence Present

code modules: lambda_runtime_ops

Security Evidence Missing

Close any unresolved ownership or unmapped threat-linkage gaps in Traceability View.

Secrets and Configuration (FU-SECRETS-CONFIGURATION)

Enforces credential isolation, least-privilege access, and redaction-safe metadata handling.

Attack Surface

none

Main Threats

Secret Leakage in Review Output

Expected Controls

Define control expectations in authored security narratives and mapped dependencies.

Security Evidence Present

code modules: secrets_configuration

Security Evidence Missing

Close any unresolved ownership or unmapped threat-linkage gaps in Traceability View.

9. Traceability View

Show requirement-to-unit-to-evidence traceability, including coverage confidence and explicit evidence gaps.

9.1. What This View Answers

- Which requirements map to which units?
- Which authored units have runtime/code evidence and which do not?
- Where are confidence or completeness gaps in trace coverage?

9.2. Coverage Gaps

Coverage summary: 7/7 units have direct evidence coverage in this view.

- Lambda Runtime Operations has no direct inferred runtime evidence yet.
- PR Context Assembly has no direct inferred runtime evidence yet.
- Policy Checks has no direct inferred runtime evidence yet.
- Secrets and Configuration has no direct inferred runtime evidence yet.

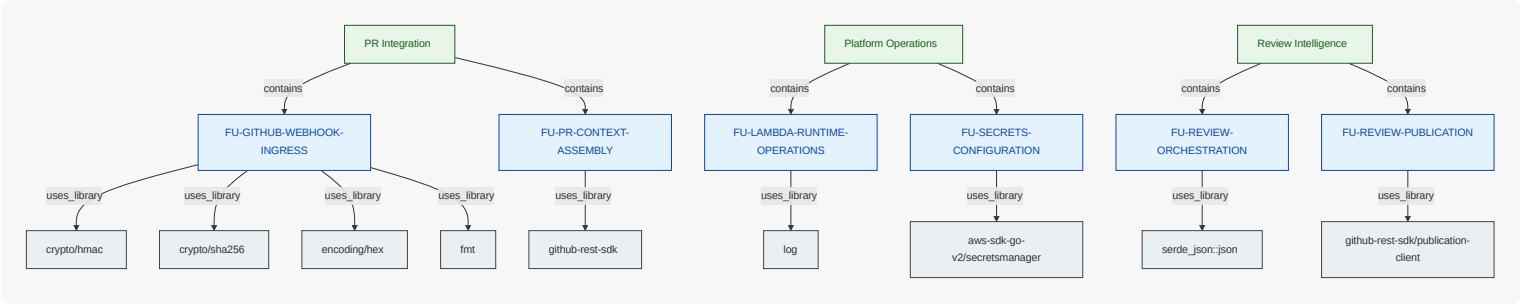
9.3. Recommended Next Evidence Additions

- Add explicit requirement-to-owner mappings where requirement scope is currently broad or ambiguous.
- Add ownership annotations for runtime and code artifacts where coverage is still unresolved.
- Prioritize closing gaps for units that are authored but lack both runtime and code evidence.

9.4. Traceability Evidence Graph

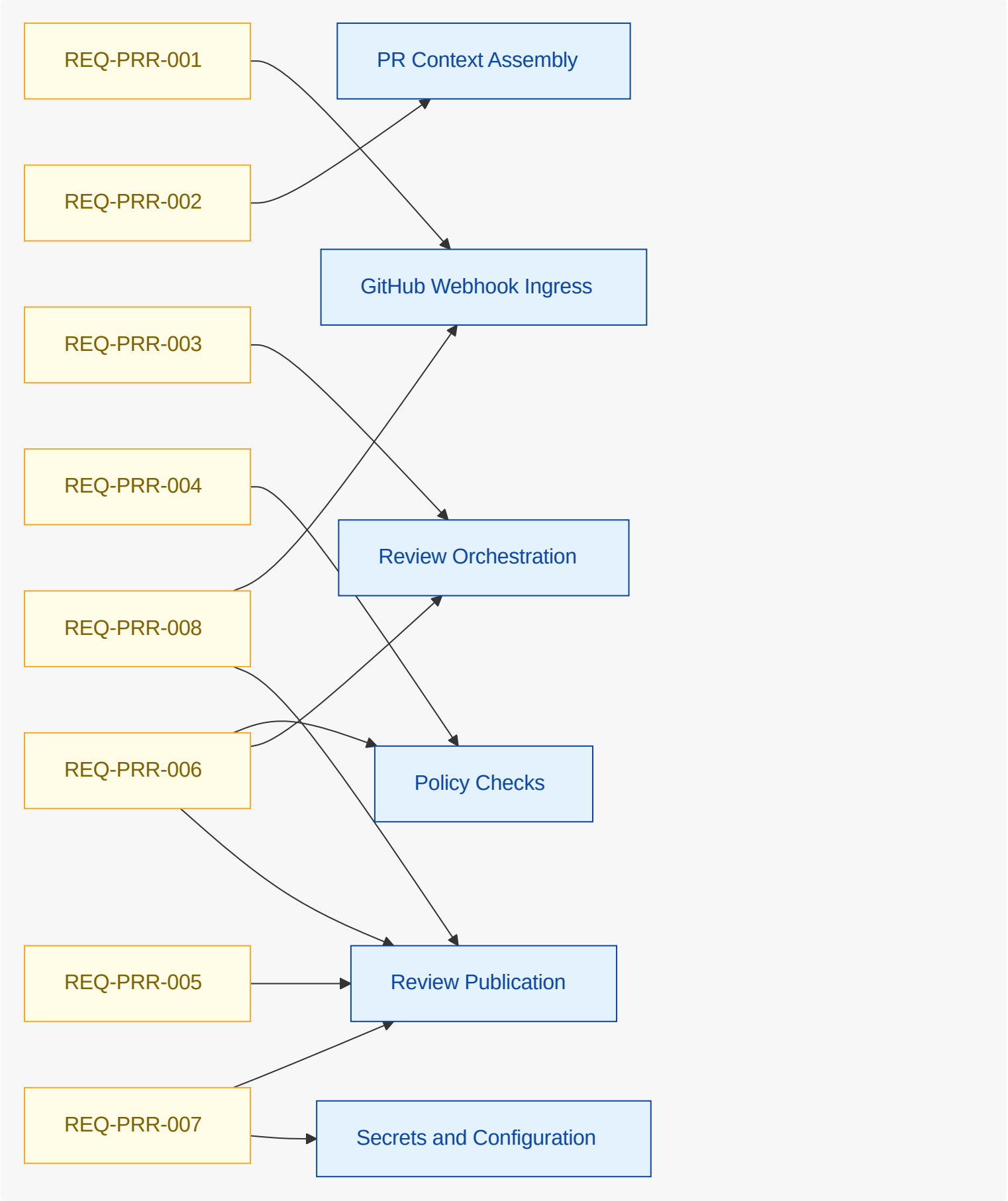
What this diagram shows: authored units and the inferred evidence links that support traceability confidence.

> **NOTE** This graph summarizes inferred ownership/evidence links and is intended for traceability confidence checks.



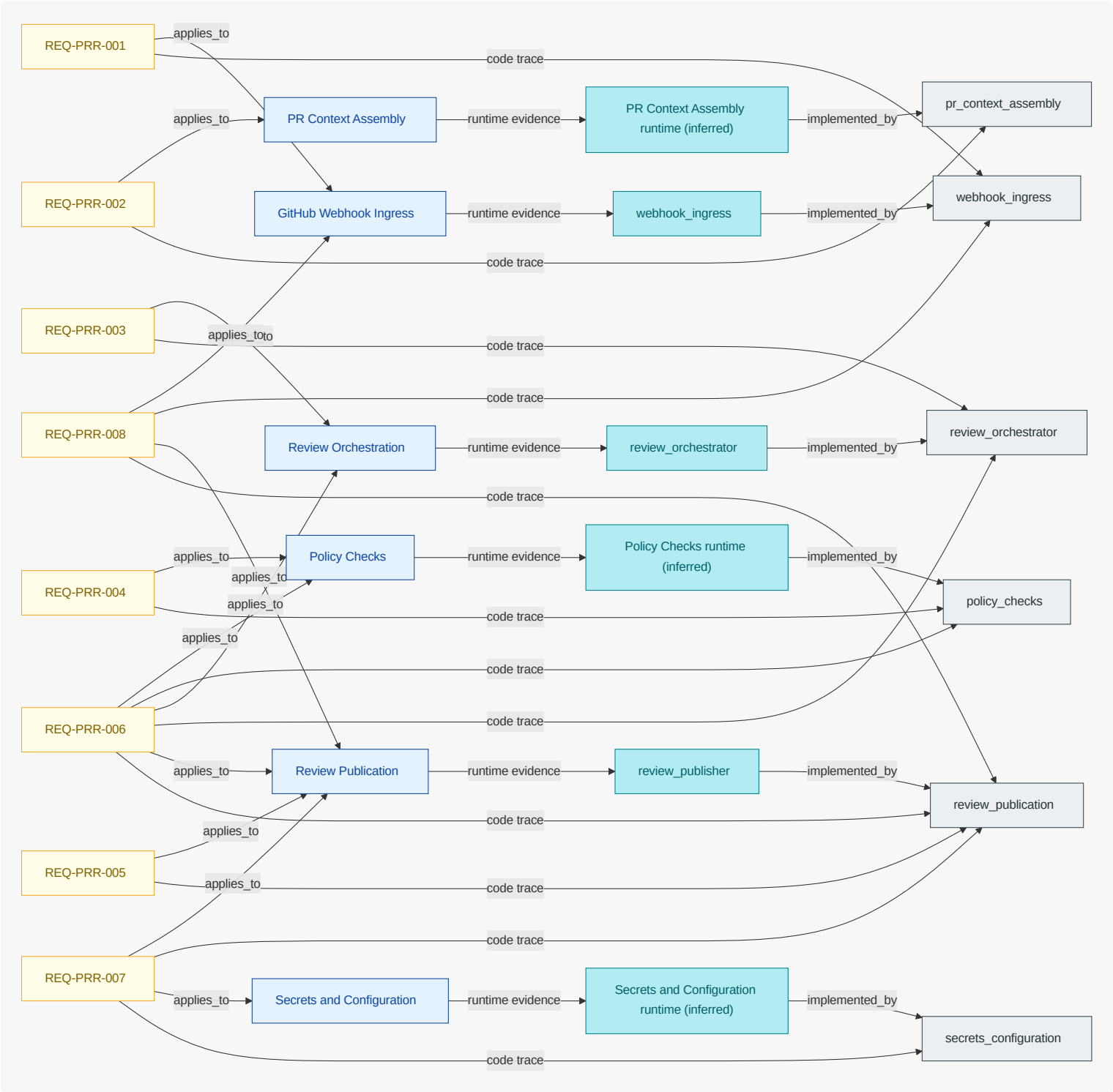
9.5. Requirement-to-Unit Mapping

What this diagram shows: direct authored mapping from requirements to the functional units they apply to.


REQ-PRR-001
REQ-PRR-002
REQ-PRR-003
REQ-PRR-004
REQ-PRR-008
REQ-PRR-006
REQ-PRR-005
REQ-PRR-007
PR Context Assembly
GitHub Webhook Ingress
Review Orchestration
Policy Checks
Review Publication
Secrets and Configuration

9.6. Requirement-to-Unit-to-Evidence Coverage

What this diagram shows: requirement-to-unit links extended with inferred runtime/code evidence attached to each unit.



Show requirement-to-unit mappings inferred from appliesTo and authored architecture ownership boundaries.

10. Traceability Appendix

10.1. Requirement Details

10.1.1. REQ-PRR-001

While pull request review is active, when pull request synchronize event is received, the pr review system shall verify webhook signature before processing review workflow. Notes: Webhook trust boundary validation.

10.1.2. REQ-PRR-002

While pull request review is active, when pull request opened event is received, the pr review system shall fetch changed files and construct diff context for review execution. Notes: Context ingestion from GitHub API.

10.1.3. REQ-PRR-003

Where ai review is enabled, when diff context is prepared, the pr review system shall request bedrock runtime analysis for security, architecture, and maintainability findings. Notes: AI-assisted review path.

10.1.4. REQ-PRR-004

Where policy checks are enabled, when pull request review is active, the pr review system shall evaluate deterministic policy checks for secret exposure, infrastructure risk, and dependency changes. Notes: Deterministic non-LLM review checks.

10.1.5. REQ-PRR-005

While pull request review is active, when review findings are generated, the pr review system shall publish check run summary and inline comments to repository maintainer. Notes: Result publication contract.

10.1.6. REQ-PRR-006

If bedrock runtime is unavailable, then the pr review system shall publish deterministic policy-only review outcome. Notes: Graceful degradation requirement.

10.1.7. REQ-PRR-007

Where audit logging is enabled, when review findings are published, the pr review system shall persist redacted prompt and result metadata for audit traceability. Notes: Evidence and audit lineage.

10.1.8. REQ-PRR-008

If review request rate limit is exceeded, then the pr review system shall defer processing and notify repository maintainer. Notes: Abuse and capacity protection behavior.

10.2. Verification Inventory

What this section shows: verification checks inferred from test artifacts, linked to test code elements and mapped to verified requirements.

10.2.1. Pr Opened Review Flow

Field	Value
ID	VER-INF-E2E-PR_OPENED_REVIEW_FLOW-6BF0B5

Field	Value
Kind	e2e
Status	pass
Verifies Requirements	REQ-PRR-001, REQ-PRR-002, REQ-PRR-003, REQ-PRR-005
Test Code Elements	tests/e2e/pr_opened_review_flow.yaml
Derived Functional Owners	FU-GITHUB-WEBHOOK-INGRESS, FU-PR-CONTEXT-ASSEMBLY, FU-REVIEW-ORCHESTRATION, FU-REVIEW-PUBLICATION
Result Summary	pass:4
Evidence	test-results/contract-checkrun.json, test-results/e2e-pr-opened-review.xml, tests/e2e/pr_opened_review_flow.yaml
Notes	Inferred from test source artifact.

10.2.2. Audit Redaction Test

Field	Value
ID	VER-INF-INTEGRATION-AUDIT_REDACTION_TEST-B108A1
Kind	integration
Status	pass
Verifies Requirements	REQ-PRR-007
Test Code Elements	tests/integration/audit_redaction_test.ts
Derived Functional Owners	FU-REVIEW-PUBLICATION, FU-SECRETS-CONFIGURATION
Result Summary	pass:1
Evidence	test-results/audit-redaction-report.md, tests/integration/audit_redaction_test.ts
Notes	Inferred from test source artifact.

10.2.3. Policy Only Fallback Test

Field	Value
ID	VER-INF-INTEGRATION-POLICY_ONLY_FALLBACK_TEST-76E20A
Kind	integration

Field	Value
Status	pass
Verifies Requirements	REQ-PRR-004, REQ-PRR-006
Test Code Elements	tests/integration/policy_only_fallback_test.rs
Derived Functional Owners	FU-POLICY-CHECKS, FU-REVIEW-ORCHESTRATION, FU-REVIEW-PUBLICATION
Result Summary	pass:2
Evidence	test-results/integration-policy-fallback.json, tests/integration/policy_only_fallback_test.rs
Notes	Inferred from test source artifact.

10.2.4. Github Check Run Contract Test

Field	Value
ID	VER-INF-TEST-GITHUB_CHECK_RUN_CONTRACT_TEST-17DBA6
Kind	test
Status	not-run
Verifies Requirements	REQ-PRR-005
Test Code Elements	tests/contract/github_check_run_contract_test.go
Derived Functional Owners	FU-REVIEW-PUBLICATION
Result Summary	not-run:1
Evidence	tests/contract/github_check_run_contract_test.go
Notes	Inferred from test source artifact.

10.2.5. Rate Limit Deferral Test

Field	Value
ID	VER-INF-UNIT-RATE_LIMIT_DEFERRAL_TEST-DC9EB2
Kind	unit
Status	pass
Verifies Requirements	REQ-PRR-008

Field	Value
Test Code Elements	tests/unit/rate_limit_deferral_test.go
Derived Functional Owners	FU-GITHUB-WEBHOOK-INGRESS, FU-REVIEW-PUBLICATION
Result Summary	pass:1
Evidence	test-results/unit-rate-limit.xml, tests/unit/rate_limit_deferral_test.go
Notes	Inferred from test source artifact.

10.3. Verification Result Mapping

What this section shows: per-requirement result mapping inferred from test-result artifacts.

Check ID	Requirement	Result	Evidence / Notes
VER-INF-E2E-PR_OPENED_REVIEW_FLOW-6BF0B5	REQ-PRR-001	pass	test-results/e2e-pr-opened-review.xml
VER-INF-E2E-PR_OPENED_REVIEW_FLOW-6BF0B5	REQ-PRR-002	pass	test-results/e2e-pr-opened-review.xml
VER-INF-E2E-PR_OPENED_REVIEW_FLOW-6BF0B5	REQ-PRR-003	pass	test-results/e2e-pr-opened-review.xml
VER-INF-INTEGRATION-POLICY_ONLY_FALLBACK_TEST-76E20A	REQ-PRR-004	pass	test-results/integration-policy-fallback.json deterministic policy checks executed
VER-INF-E2E-PR_OPENED_REVIEW_FLOW-6BF0B5	REQ-PRR-005	pass	test-results/e2e-pr-opened-review.xml
VER-INF-TEST-GITHUB_CHECK_RUN_CONTRACT_TEST-17DBA6	REQ-PRR-005	not-run	none No parsed test-result artifact for this requirement yet.
VER-INF-INTEGRATION-POLICY_ONLY_FALLBACK_TEST-76E20A	REQ-PRR-006	pass	test-results/integration-policy-fallback.json policy-only fallback published
VER-INF-INTEGRATION-AUDIT_REDACTION_TEST-B108A1	REQ-PRR-007	pass	test-results/audit-redaction-report.md
VER-INF-UNIT-RATE_LIMIT_DEFERRAL_TEST-DC9EB2	REQ-PRR-008	pass	test-results/unit-rate-limit.xml

11. Reference Index

11.1. Authored References

ACT-DEVELOPER

Kind: Actor Name: Developer

Opens pull requests and receives review feedback.

ACT-PLATFORM-OPERATOR

Kind: Actor Name: Platform Operator

Operates Lambda deployment and runtime controls.

ACT-REPOSITORY-MAINTAINER

Kind: Actor Name: Repository Maintainer

Owns merge decisions and repository-level quality gates.

ACT-SECURITY-ENGINEER

Kind: Actor Name: Security Engineer

Curates policy checks and triages security findings.

AV-PR-SPAM-ABUSE

Kind: Attack Vector Name: Pull Request Spam Abuse

High-rate PR event flooding that can exhaust review capacity.

AV-PROMPT-INJECTION-IN-DIFF

Kind: Attack Vector Name: Prompt Injection in Diff

Malicious prompt-like content embedded in pull request diffs.

AV-SECRET-LEAKAGE-IN-REVIEW

Kind: Attack Vector Name: Secret Leakage in Review Output

Generated findings accidentally exposing secrets/tokens in comments.

AV-SPOOFED-GITHUB-WEBHOOK

Kind: Attack Vector Name: Spoofed GitHub Webhook

Forged webhook payload attempting unauthorized review execution.

FG-PLATFORM-OPERATIONS

Kind: Functional Group Name: Platform Operations

Lambda deployment lifecycle, runtime operations, and secrets/configuration.

FG-PR-INTEGRATION

Kind: Functional Group Name: PR Integration

Ingress and context assembly for GitHub pull request events.

FG-REVIEW-INTELLIGENCE

Kind: Functional Group Name: Review Intelligence

AI and deterministic review analysis with result publication.

FU-GITHUB-WEBHOOK-INGRESS

Kind: Functional Unit Name: GitHub Webhook Ingress

Verifies webhook signatures, validates event type, and starts review workflow routing.

FU-LAMBDA-RUNTIME-OPERATIONS

Kind: Functional Unit Name: Lambda Runtime Operations

Manages Lambda packaging/deployment lifecycle and operational health guardrails.

FU-POLICY-CHECKS

Kind: Functional Unit Name: Policy Checks

Runs deterministic checks for secrets, risky IaC changes, and repository policy constraints.

FU-PR-CONTEXT-ASSEMBLY

Kind: Functional Unit Name: PR Context Assembly

Fetches changed files and diff metadata and prepares bounded review context payloads.

FU-REVIEW-ORCHESTRATION

Kind: Functional Unit Name: Review Orchestration

Calls Bedrock models, merges AI and deterministic findings, and handles graceful degradation paths.

FU-REVIEW-PUBLICATION

Kind: Functional Unit Name: Review Publication

Publishes check-run summaries and inline comments back to GitHub pull requests.

FU-SECRETS-CONFIGURATION

Kind: Functional Unit Name: Secrets and Configuration

Manages GitHub App credentials, Bedrock config, and redaction-safe runtime configuration.

REF-AWS-BEDROCK-RUNTIME

Kind: Referenced Element Name: AWS Bedrock Runtime API

external_service_endpoint / runtime

REF-AWS-SDK-GO

Kind: Referenced Element Name: AWS SDK for Go

third_party_library / code

REF-AWS-SECRETS-MANAGER

Kind: Referenced Element Name: AWS Secrets Manager

platform_service / runtime

REF-GITHUB-APP-API

Kind: Referenced Element Name: GitHub App API

external_service_endpoint / runtime

REF-GITHUB-REST-SDK

Kind: Referenced Element Name: GitHub REST SDK

third_party_library / code

REF-POLICY-RULESET

Kind: Referenced Element Name: Review Policy Ruleset

policy_bundle / code

REQ-PRR-001

Kind: Requirement Name: REQ-PRR-001 Resolves to: canonical While pull request review is active, when pull request synchronize event is received, the pr review system shall verify webhook signature before processing review workflow.

REQ-PRR-002

Kind: Requirement Name: REQ-PRR-002 Resolves to: canonical While pull request review is active, when pull request opened event is received, the pr review system shall fetch changed files and construct diff context for review execution.

REQ-PRR-003

Kind: Requirement Name: REQ-PRR-003 Resolves to: canonical Where ai review is enabled, when diff context is prepared, the pr review system shall request bedrock runtime analysis for security, architecture, and maintainability findings.

REQ-PRR-004

Kind: Requirement Name: REQ-PRR-004 Resolves to: canonical Where policy checks are enabled, when pull request review is active, the pr review system shall evaluate deterministic policy checks for secret exposure, infrastructure risk, and dependency changes.

REQ-PRR-005

Kind: Requirement Name: REQ-PRR-005 Resolves to: canonical While pull request review is active, when review findings are generated, the pr review system shall publish check run summary and inline comments to repository maintainer.

REQ-PRR-006

Kind: Requirement Name: REQ-PRR-006 Resolves to: canonical If bedrock runtime is unavailable, then the pr review system shall publish deterministic policy-only review outcome.

REQ-PRR-007

Kind: Requirement Name: REQ-PRR-007 Resolves to: canonical Where audit logging is enabled, when review findings are published, the pr review system shall persist redacted prompt and result metadata for audit traceability.

REQ-PRR-008

Kind: Requirement Name: REQ-PRR-008 Resolves to: canonical If review request rate limit is exceeded, then the pr review system shall defer processing and notify repository maintainer.

11.2. Catalog References

ACT-DEVELOPER

Kind: Catalog Actor Name: developer Resolves to: canonical Engineer opening and updating pull requests.

ACT-PLATFORM-OPERATOR

Kind: Catalog Actor Name: platform operator Resolves to: canonical Owner of deployment/runtime operations and reliability controls.

ACT-REPOSITORY-MAINTAINER

Kind: Catalog Actor Name: repository maintainer Resolves to: canonical Owner of merge decisions and repository quality gates.

ACT-SECURITY-ENGINEER

Kind: Catalog Actor Name: security engineer Resolves to: canonical Owner of security policy checks and finding triage.

ai review enabled

Kind: Catalog Alias Name: ai review is enabled Resolves to: canonical Alias of FEAT-AI-REVIEW

audit enabled

Kind: Catalog Alias Name: audit logging is enabled Resolves to: canonical Alias of FEAT-AUDIT-LOGGING

aws sdk

Kind: Catalog Alias Name: aws sdk for go Resolves to: canonical Alias of REF-AWS-SDK-GO

bedrock api

Kind: Catalog Alias Name: bedrock runtime Resolves to: canonical Alias of REF-AWS-BEDROCK-RUNTIME

bedrock unavailable

Kind: Catalog Alias Name: bedrock runtime is unavailable Resolves to: canonical Alias of EVT-BEDROCK-UNAVAILABLE

context assembly

Kind: Catalog Alias Name: pr context assembly Resolves to: canonical Alias of FU-PR-CONTEXT-ASSEMBLY

context prepared

Kind: Catalog Alias Name: diff context is prepared Resolves to: canonical Alias of EVT-CONTEXT-PREPARED

deterministic checks

Kind: Catalog Alias Name: policy checks Resolves to: canonical Alias of FU-POLICY-CHECKS

findings generated

Kind: Catalog Alias Name: review findings are generated Resolves to: canonical Alias of EVT-FINDINGS-GENERATED

findings published

Kind: Catalog Alias Name: review findings are published Resolves to: canonical Alias of EVT-FINDINGS-PUBLISHED

github api

Kind: Catalog Alias Name: github app api Resolves to: canonical Alias of REF-GITHUB-APP-API

github integration

Kind: Catalog Alias Name: pr integration Resolves to: canonical Alias of FG-PR-INTEGRATION

github sdk

Kind: Catalog Alias Name: github rest sdk Resolves to: canonical Alias of REF-GITHUB-REST-SDK

lambda ops

Kind: Catalog Alias Name: lambda runtime operations Resolves to: canonical Alias of FU-LAMBDA-RUNTIME-OPERATIONS

maintainer

Kind: Catalog Alias Name: repository maintainer Resolves to: canonical Alias of ACT-REPOSITORY-MAINTAINER

normal mode

Kind: Catalog Alias Name: normal mode is active Resolves to: canonical Alias of MODE-NORMAL

orchestration

Kind: Catalog Alias Name: review orchestration Resolves to: canonical Alias of FU-REVIEW-ORCHESTRATION

platform ops

Kind: Catalog Alias Name: platform operator Resolves to: canonical Alias of ACT-PLATFORM-OPERATOR

policy bundle

Kind: Catalog Alias Name: policy ruleset Resolves to: canonical Alias of REF-POLICY-RULESET

policy checks enabled

Kind: Catalog Alias Name: policy checks are enabled Resolves to: canonical Alias of FEAT-POLICY-CHECKS

pr author

Kind: Catalog Alias Name: developer Resolves to: canonical Alias of ACT-DEVELOPER

pr opened

Kind: Catalog Alias Name: pull request opened event is received Resolves to: canonical Alias of EVT-PR-OPENED

pr review system

Kind: Catalog Alias Name: review intelligence Resolves to: canonical Alias of FG-REVIEW-INTELLIGENCE

pr spam

Kind: Catalog Alias Name: pull request spam abuse Resolves to: canonical Alias of AV-PR-SPAM-ABUSE

pr synchronize

Kind: Catalog Alias Name: pull request synchronize event is received Resolves to: canonical Alias of EVT-PR-SYNCHRONIZE

prompt injection

Kind: Catalog Alias Name: prompt injection in diff Resolves to: canonical Alias of AV-PROMPT-INJECTION-IN-DIFF

publication

Kind: Catalog Alias Name: review publication Resolves to: canonical Alias of FU-REVIEW-PUBLICATION

rate limit exceeded

Kind: Catalog Alias Name: review request rate limit is exceeded Resolves to: canonical Alias of EVT-RATE-LIMIT-EXCEEDED

review active

Kind: Catalog Alias Name: pull request review is active Resolves to: canonical Alias of STATE-PR-REVIEW-ACTIVE

runtime operations

Kind: Catalog Alias Name: platform operations Resolves to: canonical Alias of FG-PLATFORM-OPERATIONS

secret config

Kind: Catalog Alias Name: secrets and configuration Resolves to: canonical Alias of FU-SECRETS-CONFIGURATION

secret leakage

Kind: Catalog Alias Name: secret leakage in review output Resolves to: canonical Alias of AV-SECRET-LEAKAGE-IN-REVIEW

secrets manager

Kind: Catalog Alias Name: aws secrets manager Resolves to: canonical Alias of REF-AWS-SECRETS-MANAGER

security reviewer

Kind: Catalog Alias Name: security engineer Resolves to: canonical Alias of ACT-SECURITY-ENGINEER

signature valid

Kind: Catalog Alias Name: webhook signature is valid Resolves to: canonical Alias of COND-WEBHOOK-SIGNATURE-VALID

webhook ingress

Kind: Catalog Alias Name: github webhook ingress Resolves to: canonical Alias of FU-GITHUB-WEBHOOK-INGRESS

webhook spoofing

Kind: Catalog Alias Name: spoofed github webhook Resolves to: canonical Alias of AV-SPOOFED-GITHUB-WEBHOOK

AV-PR-SPAM-ABUSE

Kind: Catalog Attack Vector Name: pull request spam abuse Resolves to: canonical High-volume event abuse causing review capacity exhaustion.

AV-PROMPT-INJECTION-IN-DIFF

Kind: Catalog Attack Vector Name: prompt injection in diff Resolves to: canonical Malicious prompt-like content embedded in pull request code changes.

AV-SECRET-LEAKAGE-IN-REVIEW

Kind: Catalog Attack Vector Name: secret leakage in review output Resolves to: canonical Unintended credential disclosure in generated review findings.

AV-SPOOFED-GITHUB-WEBHOOK

Kind: Catalog Attack Vector Name: spoofed github webhook Resolves to: canonical Forged webhook request attempting unauthorized review execution.

COND-WEBHOOK-SIGNATURE-VALID

Kind: Catalog Condition Name: webhook signature is valid Resolves to: canonical Condition indicating webhook HMAC verification succeeded.

DATA-CHECKRUN-SUMMARY

Kind: Catalog Data Term Name: check run summary Resolves to: canonical Aggregated review result posted as a GitHub check run.

DATA-DIFF-CONTEXT

Kind: Catalog Data Term Name: diff context Resolves to: canonical Pull request file changes and metadata prepared for analysis.

DATA-REDACTED-AUDIT-METADATA

Kind: Catalog Data Term Name: redacted audit metadata Resolves to: canonical Sanitized prompt and finding metadata retained for audit.

DATA-RESULT-METADATA

Kind: Catalog Data Term Name: result metadata Resolves to: canonical Metadata describing review output characteristics for audit and reporting.

DATA-WEBHOOK-SIGNATURE

Kind: Catalog Data Term Name: webhook signature Resolves to: canonical Signature used to verify webhook authenticity.

EVT-BEDROCK-UNAVAILABLE

Kind: Catalog Event Name: bedrock runtime is unavailable Resolves to: canonical Event raised when Bedrock inference path is degraded.

EVT-CONTEXT-PREPARED

Kind: Catalog Event Name: diff context is prepared Resolves to: canonical Event indicating context assembly completed and is ready for analysis.

EVT-FINDINGS-GENERATED

Kind: Catalog Event Name: review findings are generated Resolves to: canonical Event indicating analysis output is ready for publication.

EVT-FINDINGS-PUBLISHED

Kind: Catalog Event Name: review findings are published Resolves to: canonical Event indicating review output has been published to GitHub checks/comments.

EVT-PR-OPENED

Kind: Catalog Event Name: pull request opened event is received Resolves to: canonical Event raised when a new pull request is opened.

EVT-PR-SYNCHRONIZE

Kind: Catalog Event Name: pull request synchronize event is received Resolves to: canonical Event raised when new commits are pushed to an open pull request.

EVT-RATE-LIMIT-EXCEEDED

Kind: Catalog Event Name: review request rate limit is exceeded Resolves to: canonical Event raised when PR review rate guardrails are hit.

FEAT-AI-REVIEW

Kind: Catalog Feature Name: ai review is enabled Resolves to: canonical Feature flag enabling model-assisted review.

FEAT-AUDIT-LOGGING

Kind: Catalog Feature Name: audit logging is enabled Resolves to: canonical Feature flag enabling audit metadata persistence.

FEAT-POLICY-CHECKS

Kind: Catalog Feature Name: policy checks are enabled Resolves to: canonical Feature flag enabling deterministic policy checks.

FG-PLATFORM-OPERATIONS

Kind: Catalog Functional Group Name: platform operations Resolves to: canonical Functional group for deployment lifecycle and secret/config controls.

FG-PR-INTEGRATION

Kind: Catalog Functional Group Name: pr integration Resolves to: canonical Functional group for webhook ingress and pull request context intake.

FG-REVIEW-INTELLIGENCE

Kind: Catalog Functional Group Name: review intelligence Resolves to: canonical Functional group for AI and deterministic review analysis and publication.

FU-GITHUB-WEBHOOK-INGRESS

Kind: Catalog Functional Unit Name: github webhook ingress Resolves to: canonical Validates webhook signatures and routes trusted pull request events.

FU-LAMBDA-RUNTIME-OPERATIONS

Kind: Catalog Functional Unit Name: lambda runtime operations Resolves to: canonical Manages runtime deployment and operational controls.

FU-POLICY-CHECKS

Kind: Catalog Functional Unit Name: policy checks Resolves to: canonical Evaluates deterministic repository policy checks.

FU-PR-CONTEXT-ASSEMBLY

Kind: Catalog Functional Unit Name: pr context assembly Resolves to: canonical Fetches changed files and prepares bounded review context payloads.

FU-REVIEW-ORCHESTRATION

Kind: Catalog Functional Unit Name: review orchestration Resolves to: canonical Coordinates Bedrock analysis with deterministic checks and fallback logic.

FU-REVIEW-PUBLICATION

Kind: Catalog Functional Unit Name: review publication Resolves to: canonical Publishes review findings to GitHub checks and comments.

FU-SECRETS-CONFIGURATION

Kind: Catalog Functional Unit Name: secrets and configuration Resolves to: canonical Manages credentials and secure runtime configuration.

MODE-NORMAL

Kind: Catalog Mode Name: normal mode is active Resolves to: canonical Default operating mode using AI plus deterministic checks.

REF-AWS-BEDROCK-RUNTIME

Kind: Catalog Referenced Element Name: bedrock runtime Resolves to: canonical External Bedrock runtime endpoint used for model inference.

REF-AWS-SDK-GO

Kind: Catalog Referenced Element Name: aws sdk for go Resolves to: canonical SDK dependency used for AWS runtime integrations.

REF-AWS-SECRETS-MANAGER

Kind: Catalog Referenced Element Name: aws secrets manager Resolves to: canonical Managed secret store for application credentials and config values.

REF-GITHUB-APP-API

Kind: Catalog Referenced Element Name: github app api Resolves to: canonical External GitHub API used for pull request metadata and review publication.

REF-GITHUB-REST-SDK

Kind: Catalog Referenced Element Name: github rest sdk Resolves to: canonical SDK dependency used for GitHub App REST interactions.

REF-POLICY-RULESET

Kind: Catalog Referenced Element Name: policy ruleset Resolves to: canonical Deterministic review policy bundle used during analysis.

STATE-PR-REVIEW-ACTIVE

Kind: Catalog State Name: pull request review is active Resolves to: canonical State where review processing is currently active for a pull request.

11.3. Inferred Runtime References

review_orchestrator

Kind: Runtime lambda_function Owner: FU-REVIEW-ORCHESTRATION Source: examples/bedrock-pr-review-github-app-sample/infra/terraform/main.tf

review_publisher

Kind: Runtime lambda_function Owner: FU-REVIEW-PUBLICATION Source: examples/bedrock-pr-review-github-app-sample/infra/terraform/main.tf

webhook_ingress

Kind: Runtime lambda_function Owner: FU-GITHUB-WEBHOOK-INGRESS Source: examples/bedrock-pr-review-github-app-sample/infra/terraform/main.tf

11.4. Inferred Code References

aws-sdk-go-v2/secretsmanager

Kind: Code library_external Owner: FU-SECRETS-CONFIGURATION Source: secrets_configuration.ts

github-rest-sdk

Kind: Code library_external Owner: FU-PR-CONTEXT-ASSEMBLY Source: pr_context_assembly.ts

github-rest-sdk/publication-client

Kind: Code library_external Owner: FU-REVIEW-PUBLICATION Source: review_publication.ts

serde_json::json

Kind: Code library_external Owner: FU-REVIEW-ORCHESTRATION Source: review_orchestrator.rs

crypto/hmac

Kind: Code library_stdlib Owner: FU-GITHUB-WEBHOOK-INGRESS Source: webhook_ingress.go

crypto/sha256

Kind: Code library_stdlib Owner: FU-GITHUB-WEBHOOK-INGRESS Source: webhook_ingress.go

encoding/hex

Kind: Code library_stdlib Owner: FU-GITHUB-WEBHOOK-INGRESS Source: webhook_ingress.go

fmt

Kind: Code library_stdlib Owner: FU-GITHUB-WEBHOOK-INGRESS Source: webhook_ingress.go

log

Kind: Code library_stdlib Owner: FU-LAMBDA-RUNTIME-OPERATIONS Source: lambda_runtime_ops.go

lambda_runtime_ops.go

Kind: Code source_file Owner: FU-LAMBDA-RUNTIME-OPERATIONS Source: lambda_runtime_ops.go

policy_checks.rs

Kind: Code source_file Owner: FU-POLICY-CHECKS Source: policy_checks.rs

pr_context_assembly.ts

Kind: Code source_file Owner: FU-PR-CONTEXT-ASSEMBLY Source: pr_context_assembly.ts

review_orchestrator.rs

Kind: Code source_file Owner: FU-REVIEW-ORCHESTRATION Source: review_orchestrator.rs

review_publication.ts

Kind: Code source_file Owner: FU-REVIEW-PUBLICATION Source: review_publication.ts

secrets_configuration.ts

Kind: Code source_file Owner: FU-SECRETS-CONFIGURATION Source: secrets_configuration.ts

webhook_ingress.go

Kind: Code source_file Owner: FU-GITHUB-WEBHOOK-INGRESS Source: webhook_ingress.go

CODE-BUILDDIFFCONTEXT

Kind: Code symbol Owner: FU-PR-CONTEXT-ASSEMBLY Source: pr_context_assembly.ts:14

CODE-FALLBACKPOLICYONLY

Kind: Code symbol Owner: FU-REVIEW-ORCHESTRATION Source: review_orchestrator.rs:18

CODE-FETCHCHANGEDFILES

Kind: Code symbol Owner: FU-PR-CONTEXT-ASSEMBLY Source: pr_context_assembly.ts:8

CODE-GETGITHUBAPPPRIVATEKEY

Kind: Code symbol Owner: FU-SECRETS-CONFIGURATION Source: secrets_configuration.ts:8

CODE-NOTIFYMAINTAINERDEFERRAL

Kind: Code symbol Owner: FU-REVIEW-PUBLICATION Source: review_publication.ts:26

CODE-POLICYONLYSUMMARY

Kind: Code symbol Owner: FU-POLICY-CHECKS Source: policy_checks.rs:20

CODE-PUBLISHCHECKRUN

Kind: Code symbol Owner: FU-REVIEW-PUBLICATION Source: review_publication.ts:8

CODE-RECORDREDACTEDAUDITMETADATA

Kind: Code symbol Owner: FU-REVIEW-PUBLICATION Source: review_publication.ts:21

CODE-REQUESTBEDROCKREVIEW

Kind: Code symbol Owner: FU-REVIEW-ORCHESTRATION Source: review_orchestrator.rs:8

CODE-ROUTEPULLREQUESTEVENT

Kind: Code symbol Owner: FU-GITHUB-WEBHOOK-INGRESS Source: webhook_ingress.go:24

CODE-RUNDETERMINISTICCHECKS

Kind: Code symbol Owner: FU-POLICY-CHECKS Source: policy_checks.rs:6

CODE-VERIFYSIGNATURE

Kind: Code symbol Owner: FU-GITHUB-WEBHOOK-INGRESS Source: webhook_ingress.go:14

11.5. Verification References

VER-INF-E2E-PR_OPENED_REVIEW_FLOW-6BF0B5

Kind: Verification Check Name: Pr Opened Review Flow Resolves to: canonical Kind: e2e; Status: pass; Evidence: test-results/contract-checkrun.json, test-results/e2e-pr-opened-review.xml, tests/e2e/pr_opened_review_flow.yaml

VER-INF-INTEGRATION-AUDIT_REDACTION_TEST-B108A1

Kind: Verification Check Name: Audit Redaction Test Resolves to: canonical Kind: integration; Status: pass; Evidence: test-results/audit-redaction-report.md, tests/integration/audit_redaction_test.ts

VER-INF-INTEGRATION-POLICY_ONLY_FALLBACK_TEST-76E20A

Kind: Verification Check Name: Policy Only Fallback Test Resolves to: canonical Kind: integration; Status: pass; Evidence: test-results/integration-policy-fallback.json, tests/integration/policy_only_fallback_test.rs

VER-INF-TEST-GITHUB_CHECK_RUN_CONTRACT_TEST-17DBA6

Kind: Verification Check Name: Github Check Run Contract Test Resolves to: canonical Kind: test; Status: not-run; Evidence: tests/contract/github_check_run_contract_test.go

VER-INF-UNIT-RATE_LIMIT_DEFERRAL_TEST-DC9EB2

Kind: Verification Check Name: Rate Limit Deferral Test Resolves to: canonical Kind: unit; Status: pass; Evidence: test-results/unit-rate-limit.xml, tests/unit/rate_limit_deferral_test.go